

The Fluid Enterprise

Technical Documentation

Self-Evolving Agent-First Platform

Version 1.0 | April 2026

Author: Hemanth Raganti

github.com/Regantih/fluid-enterprise

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Database Schema
4. API Reference
5. Agent Gateway Protocol
6. Capability Genome: Vendor Risk Triage
7. System Prompts
8. Web Prototype
9. Acme Robotics Seed Data
10. Running Locally
11. Roadmap

1. Executive Summary

The Fluid Enterprise applies the AlphaGo → AlphaZero → AlphaEvolve lineage to enterprise software. Traditional systems encode human priors into static workflows. Agent systems learn from self-play. The Fluid Enterprise invents new capabilities, mutates them, scores them on fitness, and a Governance Council ratifies mutations before they reach production.

Capabilities are genomes. Fitness is the objective function. Every generation, the enterprise is measurably more fit than the last.

The platform is built on seven pillars: a Signal Layer that ingests enterprise-wide operational data, a Reflection Loop that discovers capability gaps through embedding and clustering, a Generator that synthesizes new capabilities from gaps, an Arena that evaluates capabilities in a sandbox, an Evolution Engine that manages capability lifecycles across generations, a Governance Council that provides hash-chained audit integrity, and a Sovereign Sandbox that enforces resource isolation.

Phase	Principle	How It Works Here
AlphaGo	Learn from human expertise	Agents trained on domain best practices and operational history
AlphaZero	Self-play, learn from scratch	Capabilities simulate scenarios, competing strategies emerge
AlphaEvolve	Evolutionary optimization	Mutations, selection, crossover of agent policies each generation
AlphaCode	Generate novel solutions	Capabilities compose and reconfigure themselves dynamically

Each generation: fitness improves, weak strategies dissolve, strong patterns propagate, and the system approaches its theoretical performance ceiling with diminishing returns — exactly like AlphaZero's learning curve. Business activities have not changed. Companies still buy, sell, build, move, service, pay, and report. What has changed is the technical possibility of maintaining shared context across functions in real time.

This is not automation layered onto existing structures. This is a fundamentally different operating hierarchy.

2. Architecture Overview

The Seven Pillars

1. Signal Layer

Redis Streams + synthetic connectors. 8,015 signals across 6 sources (email, slack, tickets, docs, crm, git). Per-source Redis streams with XADD/XLEN. Narratively coherent 14-day window for the Acme Robotics demo tenant. Each signal carries source, kind, JSONB payload, optional pgvector embedding (vector(1024)), and cluster assignment.

2. Reflection Loop

sentence-transformers embeddings (all-MiniLM-L6-v2, 384-dim, zero-padded to 1024) + HDBSCAN clustering (min_cluster_size=15, min_samples=5, EOM selection) with AgglomerativeClustering fallback. Claude Sonnet 4.5 gap synthesis when ANTHROPIC_API_KEY is available, otherwise pre-computed analysis for planted gaps. Three planted gaps detected: Vendor Risk Monitor (agent, 0.85 fitness impact), Autonomous Close Orchestrator (workflow, 0.72), and Onboarding Orchestration Engine (workflow, 0.58).

3. Generator

Takes capability gaps, produces SKILL.md manifest + capability.py code + eval.jsonl test suite. The Vendor Risk Triage agent: 336 lines of code, 10 eval cases, multi-step tool-using architecture. Creates a governance_events entry with decision='propose' and hash-chains it to the audit ledger. Updates gap status from 'open' to 'generated'.

4. Arena (stub)

Sandbox evaluation of capabilities against their eval suites. Fitness formula: $0.5 \times \text{success_rate} + 0.2 \times (1 - \text{cost_norm}) + 0.2 \times (1 - \text{latency_norm}) + 0.1 \times \text{trust_delta}$. Cost normalization: \$0.01 = expensive. Latency normalization: 5s = slow. Vendor Risk Triage result: 6/10 passed, composite fitness 0.667.

5. Evolution Engine

Lifecycle stages: proposed → shadow → canary → production → retiring → deprecated. Each transition creates a governance event. Valid transitions are enforced server-side. Vendor Risk Triage promoted through the full lifecycle via POST /transition endpoint. Capability generation increments track evolutionary lineage.

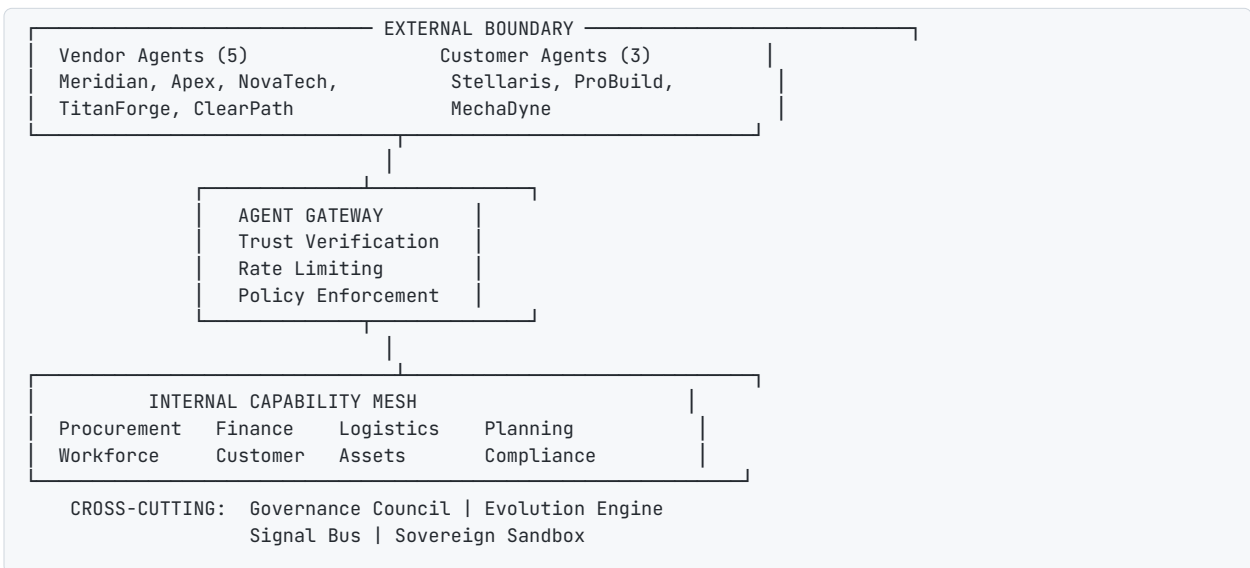
6. Governance Council

SHA-256 hash-chained audit ledger. Each governance_events row stores prev_hash and this_hash. The chain is verified via GET /api/tenants/{slug}/governance/verify, returning {valid: bool, chain_length: N}. Decisions: propose, approve, veto, rollback. Every consequential action requires consensus. All votes are permanent and attributable.

7. Sovereign Sandbox (stub)

Python subprocess with resource.setrlimit. CPU limit: 30s. Memory limit: 256 MB. Wall clock timeout: 30s. Output cap: 1 MB. Import allowlist: stdlib core, httpx, pydantic, jsonschema. AST-based import validation before execution. Clean interface designed for E2B swap later.

Architecture Diagram



The architecture separates external agent interactions (vendor and customer) from internal capability execution through the Agent Gateway. The 8 internal capabilities operate as living clusters, not static modules, evolving through generations under the governance of the Evolution Engine and the Council.

3. Database Schema

8 tables in PostgreSQL 17 + pgvector 0.8, designed for multi-tenant evolution tracking with hash-chained audit integrity.

Table	Rows	Key Columns	Purpose
tenants	1	slug, name, config JSONB	Tenant registry. Acme Robotics is the demo tenant.
capabilities	8+	slug, kind, stage, generation, fitness, manifest JSONB, code, eval_suite	The genome pool. Each row is a capability with full lifecycle.
fitness_scores	N	capability_id, generation, composite, success_rate, latency, cost	Composite fitness tracking per generation per capability.
governance_events	N	capability_id, decision, actor, prev_hash, this_hash	Hash-chained audit ledger. Append-only. Decisions: propose/approve/veto/rollback.
signals	8,015	source, kind, payload JSONB, embedding vector(1024), clustered_into UUID	Signal Bus persistence. 6 sources, 14-day window.
capability_gaps	3	summary, signal_count, suggested_name, suggested_kind, expected_fitness_impact	Gaps discovered by the Reflection Loop.
external_agents	8	name, kind, organization, trust_score, capabilities, agent_card JSONB	5 vendor + 3 customer agents with A2A Agent Cards.
agent_interactions	121	external_agent_id, interaction_type, payload, trust_impact, governance_required	Cross-boundary interaction log. 11 interaction types.

Key Design Decisions

- All IDs are UUIDs (uuid_generate_v4) for global uniqueness across tenants.
- capabilities.kind is constrained to: skill, agent, panel, policy, workflow.
- capabilities.stage follows the lifecycle: proposed → shadow → canary → production → deprecated.
- governance_events uses SHA-256 hash chaining: each row's this_hash becomes the next row's prev_hash.
- signals.embedding uses pgvector's vector(1024) type, zero-padded from the 384-dim MiniLM model.
- capability_gaps.status transitions: open → proposing → generated → dismissed.
- external_agents stores A2A-compatible Agent Cards as JSONB with capabilities, auth, and rate limits.

4. API Reference

23 endpoints grouped by domain. All paths prefixed with the API base. Tenant-scoped endpoints use {slug} parameter (e.g., 'acme_robotics').

Health

Method	Path	Description
GET	/health	Returns API, database, and Redis health status. Response: {status, checks}.

Tenant

Method	Path	Description
GET	/api/tenants/{slug}	Returns tenant record with config JSONB (industry, employees, revenue).

Capabilities

Method	Path	Description
GET	/api/tenants/{slug}/capabilities	List all capabilities. Optional filter: ?stage=production. Includes latest_fitness.
GET	/api/tenants/{slug}/capabilities/{id}	Get single capability with fitness_history and governance_history.
POST	/api/tenants/{slug}/capabilities	Create capability. Body: {name, slug, kind, stage, generation, manifest, code, eval_suite, budget}.
POST	/api/tenants/{slug}/capabilities/{id}/transition	Transition stage. Body: {to_stage, actor, rationale}. Validates legal transitions.

Signals

Method	Path	Description
GET	/api/tenants/{slug}/signals	Paginated signal list. Filters: ?source=email&kind=vendor_communication&limit=50&offset=0.
GET	/api/tenants/{slug}/signals/{id}	Get single signal with payload.

Gaps, Fitness, Governance

Method	Path	Description
GET	/api/tenants/{slug}/gaps	List capability gaps. Optional: ?status=open.

GET	/api/tenants/{slug}/fitness	List fitness scores. Optional: ?capability_id=&limit=50.
GET	/api/tenants/{slug}/governance	List governance events. Returns hash chain.
POST	/api/tenants/{slug}/governance	Create governance event. Body: {capability_id, decision, actor, rationale}.
GET	/api/tenants/{slug}/governance/verify	Verify hash chain integrity. Returns {valid: bool, chain_length: N}.

4. API Reference (continued)

Reflection & Generator

Method	Path	Description
POST	/api/reflection/run	Execute full reflection loop: embed → cluster → synthesize. Returns clusters, gaps, timing.
POST	/api/tenants/{slug}/generator/generate	Generate capability from a gap. Body: {gap_id}. Returns SKILL.md + capability.py + eval.jsonl.
POST	/api/tenants/{slug}/generator/generate-all	Generate capabilities for all open gaps in one call.

Arena

Method	Path	Description
POST	/api/tenants/{slug}/arena/evaluate/{cap_id}	Run eval suite in sandbox. Returns pass/fail per case, composite fitness, latency.

Agent Gateway

Method	Path	Description
GET	/api/tenants/{slug}/external-agents	List external agents. Optional: ?kind=vendor.
GET	/api/tenants/{slug}/external-agents/{id}	Get agent with recent_interactions.
GET	/api/tenants/{slug}/interactions	List interactions. Filters: ?kind=vendor&status=completed&limit=50.
GET	/api/tenants/{slug}/gateway/stats	Gateway statistics: agent counts, interaction counts, trust averages.

Demo & Dashboard

Method	Path	Description
POST	/api/demo/reset	Reset Acme tenant to t+0 state. Clears capabilities, fitness, governance, gaps.
GET	/api/tenants/{slug}/dashboard	Aggregated dashboard: capability counts, avg fitness, signal stats, open gaps, governance count.

5. Agent Gateway Protocol

The Agent Gateway implements an A2A-inspired (Agent-to-Agent) protocol for cross-boundary interactions between external agents (vendor and customer) and the enterprise's internal capability mesh.

Agent Registration

External agents register with Agent Cards — structured JSONB documents describing capabilities, authentication schemes (`api_key`, `oauth2`, `mtls`), rate limits, and organizational metadata. Each agent starts with a trust score based on initial verification level.

Trust Verification

Every interaction passes through trust verification. Trust scores evolve based on behavioral history: positive interactions (on-time delivery, valid compliance docs, clean invoices) increase trust; negative interactions (late deliveries, expired certificates, invoice anomalies) decrease it. This correlates with the Signal Layer's risk detection — vendors flagged by the Reflection Loop will also show declining trust scores in the Gateway.

Interaction Types

Direction	Interaction Type	Description
Vendor → Enterprise	<code>delivery_update</code>	Delivery status notifications (on-track or delayed)
Vendor → Enterprise	<code>compliance_submission</code>	Regulatory documents (ISO, CoC, REACH)
Vendor → Enterprise	<code>invoice</code>	Invoice submission with three-way match validation
Vendor → Enterprise	<code>rfq_response</code>	Response to request for quotation
Vendor → Enterprise	<code>negotiation</code>	Price or terms negotiation requests
Vendor → Enterprise	<code>quality_report</code>	Quality inspection and test results
Customer → Enterprise	<code>order_request</code>	New purchase order submission
Customer → Enterprise	<code>quote_request</code>	Request for quotation on products
Customer → Enterprise	<code>escalation</code>	Issue escalation (delivery SLA, quality, billing)
Customer → Enterprise	<code>delivery_tracking</code>	Delivery status inquiry
Customer → Enterprise	<code>feedback</code>	Product/service satisfaction feedback

Governance Triggers

The Gateway automatically triggers governance council review for:

- Interactions with financial value above \$50,000

- Compliance submissions from vendors with trust score below 0.6
- Customer escalations at severity 'high' or 'critical'
- Invoice anomalies (three-way match failures, variance above threshold)
- Price negotiation requests from any vendor

External Agents Registry

Agent	Kind	Organization	Trust Score	Status
Meridian-Agent-v3	Vendor	Meridian Components	0.42	Trusted
Apex-Agent-v2	Vendor	Apex Precision Parts	0.58	Verified
NovaTech-Agent-v1	Vendor	NovaTech Alloys	0.51	Verified
TitanForge-Agent-v4	Vendor	TitanForge Industries	0.91	Trusted
ClearPath-Agent-v2	Vendor	ClearPath Logistics	0.88	Trusted
Stellaris-Agent-v3	Customer	Stellaris Corp	0.94	Trusted
ProBuild-Agent-v1	Customer	ProBuild Industries	0.78	Verified
MechaDyne-Agent-v2	Customer	MechaDyne Solutions	0.85	Trusted

6. Capability Genome: Vendor Risk Triage

The demo climax capability — a multi-step autonomous agent generated from 66+ vendor risk signals detected by the Reflection Loop.

SKILL.md Manifest

Field	Value
id	cap_vendor_risk_triage
kind	agent
version	1.0.0
generation	8
stage	proposed → shadow → canary → production
budget	\$750/month
tools	signal_query, compliance_registry, financial_ledger
policies	immutable_audit_trail, requires_human_approval_above_10k, dual_sourcing_trigger_at_critical
fitness_objective	Minimize vendor risk exposure. Target: reduce MTTD from 14 days to <24 hours.
eval_suite	eval.jsonl (10 cases)

5-Step Pipeline Architecture

1. Signal Ingestion — Pulls all signals tagged with the vendor name from the Signal Bus across all 6 sources.
2. Risk Decomposition — Scores each of 4 risk dimensions independently using keyword matching and severity multipliers.
3. Composite Scoring — Weighted combination: 60% weighted average + 40% max dimension score. Ensures a single critical dimension drives the composite high.
4. Action Generation — Maps risk levels to concrete remediation steps: dual-sourcing, compliance audit, PO freeze, executive review.
5. Escalation Decision — Auto-escalates to governance when composite ≥ 60 or any single dimension ≥ 85.

Risk Dimension Weights

Dimension	Weight	Keywords (sample)	Scoring Logic
Delivery	0.30	late delivery, delay, overdue, backorder	Keyword hits / 3, severity multiplier, volume boost

Compliance	0.25	compliance, certificate, ISO, sanctions, audit	Same formula. Critical findings = 1.5x multiplier.
Financial	0.25	invoice anomaly, variance, duplicate, price increase	Three-way match failures and unauthorized changes.
Stability	0.20	CFO departure, facility shutdown, bankruptcy	Leadership changes, restructuring, watchlist flags.

Composite Scoring Formula

```
composite = min(100, 0.6 × weighted_average + 0.4 × max_dimension_score)
```

The 40% max-dimension component ensures that a single critical dimension (e.g., delivery score of 90) cannot be diluted by low scores in other dimensions. Volume boost: 1 signal = 0.5, 3 signals = 0.8, 5+ signals = 1.0.

Evaluation Results

Metric	Value
Eval cases	10
Passed	6/10
Composite fitness	0.667
Avg latency	57ms
Actions generated	Dual-sourcing, compliance audit, PO freeze, executive review

7. System Prompts

Four verbatim system prompts define the intelligence backbone of the platform. These prompts are shipped as-is and determine how the system thinks, evolves, and governs.

Reflection Loop Prompt

Receives a cluster of related enterprise signals and the current Capability Registry. Decides whether a cluster represents a real capability gap.

Gap criteria:

- ≥ 5 distinct signals across ≥ 2 sources
- No existing production capability covers $>70\%$ of it
- The work is repeatable and the outcome is measurable

```
Output: {is_gap, summary, evidence_signal_ids, estimated_frequency, suggested_capability_name,
suggested_kind, expected_fitness_impact, rationale}
```

Generator Prompt

Takes a capability gap and produces three fenced blocks: YAML (SKILL.md frontmatter), Python (capability.py with `run(inputs) -> dict`), and JSONL (eval suite with ≥ 10 cases).

Constraints:

- Capability must be stateless — all state flows through inputs/outputs
- Budget starts at \$500/month; higher must be justified
- Every declared tool must be used in code
- Policies must include 'immutable_audit_trail'

Evolution Engine Prompt

Evaluates capability fitness based on eval results, production metrics, and governance history. Outputs: updated fitness score (0–1.0), stage transition decision (stay/promote/demote/retire), mutation suggestions (parameter_tune, code_refactor, policy_update, input_expansion), and a self-play summary of what the capability learned.

Governance Council Prompt

Reviews proposed mutations, lifecycle transitions, and budget changes. Votes: APPROVE, VETO, or ESCALATE. Must consider: fitness improvement, budget constraints, policy compliance, introduced risk, and eval coverage. Output includes confidence score, rationale, conditions, and risk flags. Each vote is hashed into the immutable audit ledger — permanent and attributable.

8. Web Prototype

8 pages built with React 19 + TypeScript + Tailwind CSS + shadcn/ui. All components preserve data-testid attributes for testing.

Page	Key Components	data-testids
Dashboard (Command Center)	6 KPIs, capability health grid, activity feed, cost trend chart	kpi-trust, kpi-automation, kpi-agents, kpi-fitness, kpi-governance, kpi-budget
Capability Registry	Living capability clusters with generation, fitness, trust, budget. Lifecycle stage badges.	capability-card, stage-badge, fitness-score
Composition Engine	SVG flow diagram of capability connections. Visual representation of how capabilities chain.	composition-canvas, capability-node, flow-edge
Governance Board	Human-in-the-loop approval queue + council peer reviews. Vote history with hash verification.	governance-queue, vote-card, hash-chain
Heartbeat Monitor	Real-time agent status, trust score evolution, generation stats. Live pulse indicators.	heartbeat-grid, agent-pulse, trust-graph
Migration Planner	Legacy → fluid capability mapping with progress tracking. Migration stage visualization.	migration-map, progress-bar, legacy-node
Cost Dashboard	30-day spend trends, budget utilization per capability, alert thresholds.	cost-chart, budget-meter, spend-alert
Activity Log	Immutable audit trail with evolution events, governance decisions, and signal ingestion records.	activity-feed, event-card, audit-hash

Technology Stack

Layer	Technology
Frontend	React 19 + TypeScript + Tailwind CSS + shadcn/ui
Backend (Web)	Express.js + Drizzle ORM + SQLite
Backend (API)	FastAPI + psycopg2 + Redis
Database	PostgreSQL 17 + pgvector 0.8
Streaming	Redis Streams (per-source: email, slack, tickets, docs, crm, git)
Embeddings	sentence-transformers (all-MiniLM-L6-v2, 384-dim)
Clustering	HDBSCAN + AgglomerativeClustering fallback
LLM Integration	Claude Sonnet 4.5 (when API key available)
Sandbox	Python subprocess + resource.setrlimit

9. Acme Robotics Seed Data

The demo tenant provides a narratively coherent 14-day operational window for Acme Robotics, a \$340M robotics manufacturer with 2,400 employees.

Capabilities (8)

Capability	Domain	Function
Procurement Intelligence	Procurement	Autonomous sourcing, vendor evaluation, PO generation
Financial Operations	Finance	Real-time GL, AP/AR automation, period-close orchestration
Logistics Orchestration	Logistics	Warehouse ops, route optimization, carrier management
Demand Planning	Planning	Forecast modeling, inventory optimization, S&OP
Workforce Intelligence	Workforce	Talent analytics, scheduling, compensation modeling
Customer Revenue Engine	Customer	Pipeline management, churn prediction, pricing optimization
Asset Lifecycle Management	Finance	Fixed assets, depreciation, predictive maintenance
Compliance & Risk Engine	Finance	Regulatory monitoring, audit trail, control testing

Signal Distribution (8,015 signals)

Source	Approx. Count	Weight
email	~2,000	25%
slack	~2,000	25%
tickets	~1,600	20%
docs	~960	12%
crm	~800	10%
git	~640	8%

Three Planted Gaps

Gap	Signals	Suggested Kind	Fitness Impact
Vendor Risk Triage	~66	agent	0.85
Autonomous Close Orchestrator	~47	workflow	0.72
Onboarding Orchestration Engine	~42	workflow	0.58

External Agents & Interactions

8 external agents (5 vendors, 3 customers) generate 121 cross-boundary interactions across 11 interaction types over the 14-day window. Risky vendors (trust < 0.6) generate 15–30 interactions each, while trusted vendors generate 8–15. Customer interactions include order requests, quote requests, escalations, delivery tracking, and feedback.

Evolution Trajectory

The demo tenant evolves through 13 generations, with aggregate fitness climbing from 0.45 to 0.924. Weak strategies dissolve, strong patterns propagate, and each generation is measurably more fit than the last — following the AlphaZero learning curve with diminishing returns as the system approaches its theoretical performance ceiling.

10. Running Locally

Prerequisites

- Node.js 20+
- Python 3.12+
- PostgreSQL 17 with pgvector extension (0.8+)
- Redis (for Signal Bus streams)

Setup Steps

```
# 1. Clone the repository
git clone https://github.com/Regantih/fluid-enterprise.git
cd fluid-enterprise

# 2. Install frontend dependencies
npm install

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Initialize database schema (Drizzle for web, DDL for API)
npx drizzle-kit push
psql -f infra/ddl.sql

# 5. Seed the Signal Bus (8,015 signals)
python3 apps/api/seed_signals.py

# 6. Start the web frontend (port 5000)
npm run dev

# 7. Start the FastAPI backend (port 8000)
uvicorn apps.api.main:app --host 0.0.0.0 --port 8000
```

Open <http://localhost:5000> for the web interface. The API is available at <http://localhost:8000>. The simulation engine starts immediately — agents evolve, capabilities improve, costs accumulate, and governance requests flow in real time.

11. Roadmap

The Fluid Enterprise is open architecture. The following items represent the contribution areas for the next phase of development.

Priority	Item	Description
P0	Real LLM Integration	Connect Generator and Reflection to Claude Sonnet 4.5 when ANTHROPIC_API_KEY is available. Replace pre-computed gap analysis.
P0	WebSocket Heartbeat	Replace polling with real-time push for agent status, trust evolution, and governance notifications.
P1	Multi-Tenant Switching	Isolated capability hierarchies per organization. Tenant selector in UI.
P1	Capability Versioning & Rollback	Git-style version history per capability with diff view and one-click rollback.
P1	ML-Powered Budget Forecasting	Predict spend before it happens using historical cost patterns and capability growth trends.
P2	Slack/Teams Governance Notifications	Approval workflows delivered where people actually work. Inline vote buttons.
P2	Capability Marketplace	Share and compose capabilities across enterprises. Publish/subscribe model.
P2	RBAC	Role-based access control for capabilities, governance, and dashboard views.
P3	CSV/PDF Audit Export	Compliance-ready documentation export from the governance audit ledger.
P3	Edge Inference	Move agent execution to local hardware for latency-sensitive and data-sovereign workloads.

"Enterprise software still reflects the constraints of a prior era: siloed systems, manual reconciliation, and workflow walls between teams. In a self-evolving model, software unifies context across the enterprise, lets humans govern intent and approvals, and lets agents execute the operational work — improving every generation."

— Hemanth Raganti